Payouts API

High-Level Design

Boku — Banking & Settlement

Senior Backend Engineer take-home · 2026

The Task

> Design a Payouts API that lets a caller submit a payout to a beneficiary, exposes clear endpoints and request/response contracts, handles duplicate requests / failed disbursements / partial failures, and lets the caller track status after submission.

WHAT I CHOSE TO DEMONSTRATE

The **Pull-and-Pay funding model with a compliance gate** — the most complex realistic flow. It forces the full design: async compliance, explicit fund pull state, ambiguous-failure handling, and a refund path. More representative of a real platform than the simpler prefunded model.

4 Assumptions

Each resolves a scenario the task leaves open.

#	Assumption	Key trade-off
1	**Submission sync, settlement async**	`202 Accepted` + async tracking vs. coupling API uptime to rail latency
2	**Single payout = primary unit; batch is additive**	Get idempotency right on one payout first, then compose
3	**Platform picks the rail, caller doesn't**	Hides internal topology; Boku's value prop is the 65-country coverage
4	**Pull-and-Pay as primary showcase**	Forces full design: compliance gate, fund pull state, refund flow

Assumption 1 — Async by Design

Scenario: `POST /payouts` hits the API. The actual funds movement can take seconds to multiple business days depending on corridor and rail.

Decision: `202 Accepted` immediately. Acceptance path = thin DB write + fire-and-forget Kafka publish, **< 50ms P95** — comfortably inside any API Gateway timeout.

Alternative rejected: Fully synchronous confirm-then-respond — couples API availability to partner rail latency. One slow rail makes the entire API appear down.

```
POST /payouts          →   202 Accepted  { payout_id, status: PENDING }
                               ↓
                         async pipeline
                               ↓
GET /payouts/{id}      →   { status: SETTLED, history: [...] }
```

Assumption 3 — Rail Selection Is the Platform's Job

Scenario: Boku is global (65 countries) — Singapore, Kenya, and the US ride completely different rails.

Decision: Caller specifies **what** and **to whom**. Platform decides **how**.

Caller-selects-rail	Platform-selects-rail (chosen)
Explicit `"rail": "LOCAL_BANK_SG"` in request	`amount`, `currency`, `beneficiary` only
Leaks internal partner topology into public contract	Rail selection + failover internal
Couples API to infra decisions	Caller API stays simple

> **Open question Q1:** Is Boku a managed service (platform picks rail) or a connectivity layer (caller picks rail)? If the latter, the design simplifies — rail selection layer disappears entirely.

Assumption 4 — Pull-and-Pay as the Showcase

Model	How it works	Design impact
Prefunded wallet	Merchant tops up Boku balance in advance	Synchronous `INSUFFICIENT_FUNDS`. Simpler — no refund flow.
Pull-and-Pay (primary)	Platform debits caller via direct debit mandate as a dedicated async step	Two failure surfaces. Full design: compliance gate, `FUND_PULLING` state, refund flow.

Why Pull-and-Pay forces the right design:

- Compliance gate must run **before** any funds move
- `FUND_PULLING` as a separate state means reaching `SUBMITTED` **guarantees** funds are with Boku — no `funds_pulled` flag needed anywhere
- Every failure from `SUBMITTED` triggers a refund unconditionally

> **Open question Q3:** Which model does Boku actually use? If prefunded, the design simplifies: remove `FUND_PULLING`, `FUND_PULL_FAILED`, and the refund flow.

API Surface

Endpoint	Purpose
`POST /payouts`	Submit a single payout — `Idempotency-Key` required
`GET /payouts/{id}`	Current status + full lifecycle history — always the source of truth
`GET /payouts?external_ref={ref}`	Recovery path when caller crashed before persisting `payout_id`
`POST /payouts/batch`	Fan-out to N independent payout records
`GET /payouts/batch/{batchId}`	Rollup: `succeeded/failed/pending` counts
`POST /payouts/{id}/cancel`	State-machine guard — only `PENDING` is cancellable
`POST /webhooks/subscriptions`	Register callback URL — push complement to polling
`POST /quotes`	Lock FX rate before payout — optional, short TTL
`GET /quotes/{quoteId}`	Check quote validity

`DELETE /payouts/{id}` is absent — funds-movement records are never deleted, only transitioned to terminal states.

Payout State Machine

```
PENDING → PENDING_COMPLIANCE → FUND_PULLING → SUBMITTED → SETTLED    ← happy path
PENDING → FUND_PULLING → SUBMITTED → SETTLED                          ← no compliance gate
PENDING → SUBMITTED → SETTLED                                         ← prefunded wallet

PENDING_COMPLIANCE → REJECTED_COMPLIANCE                              ← hard AML hit (terminal, no funds moved)
PENDING_COMPLIANCE → PENDING_MANUAL_REVIEW                            ← soft hit (human review)
PENDING_MANUAL_REVIEW → FUND_PULLING | REJECTED_COMPLIANCE            ← reviewer decides

FUND_PULLING → SUBMITTED                                              ← pull succeeds, funds with Boku
FUND_PULLING → FUND_PULL_FAILED                                       ← terminal, no refund needed

SUBMITTED → SETTLED | FAILED | RETURNED | SUBMITTED_UNCONFIRMED
SUBMITTED_UNCONFIRMED → SETTLED | FAILED                              ← partner-ref lookup only

FAILED   → REFUND_PENDING → REFUNDED | REFUND_FAILED
RETURNED → REFUND_PENDING → REFUNDED | REFUND_FAILED

PENDING → CANCELLED
```

Two-layer write guard — not just one:

`UPDATE ... WHERE status = 'X'` handles duplicate Kafka events on the same worker. But it doesn't cover **two independent writer types** reading the same row simultaneously before either commits — e.g. a partner callback + midnight batch reconciliation both seeing `SUBMITTED` and both writing `SETTLED`.

Optimistic locking with a `version` column closes this gap:

Compliance Gate — Before Any Funds Move

```
payout.created (Kafka)
        ↓
  Fraud screening          ← velocity, amount anomaly, beneficiary pattern
      PASS → AML scan      ← OFAC, UN, EU, local sanctions + PEP lists
          PASS  → payout.compliance_passed → FUND_PULLING
          HARD HIT → REJECTED_COMPLIANCE (terminal, auto, may trigger regulatory reporting)
          SOFT HIT → PENDING_MANUAL_REVIEW (human review, SLA-bound)
      FAIL → REJECTED_COMPLIANCE (terminal)
```

Why `REJECTED_COMPLIANCE` ≠ `FAILED`:

- `FAILED` = rail issue, potentially retryable with a different account
- `REJECTED_COMPLIANCE` = legal/policy block — no retry changes the outcome; may trigger reporting obligation

Why compliance runs before fund pull:

No money moves until the payout is clean. `REJECTED_COMPLIANCE` requires no refund — nothing was debited.

The `SUBMITTED_UNCONFIRMED` Problem

Scenario: Disbursement worker sends to Partner Rail. Connection drops. Did the partner process it?

```
DW → Rail: submit disbursement (client_reference = Idempotency-Key)
Rail: [timeout / network error]
DW → DB: UPDATE status = SUBMITTED_UNCONFIRMED
```

The wrong responses:

- ❌ Trigger refund immediately → refund money that was already disbursed
- ❌ Failover to Partner B → **double payment** if Partner A processed it

The right response:

```
DW → Rail: GET /status?ref=<Idempotency-Key>
         SETTLED → update SETTLED, notify merchant
         NOT FOUND / FAILED → update FAILED → trigger refund
```

`SUBMITTED_UNCONFIRMED` is deliberately non-terminal. Failover is blocked until the partner-reference lookup resolves the outcome.

Refund Trigger — Simplified by `FUND_PULLING` Separation

State	Refund needed?	Why
`REJECTED_COMPLIANCE`	**No**	Compliance runs before any funds move
`FUND_PULL_FAILED`	**No**	Caller's account was never debited
`FAILED` from `SUBMITTED`	**Yes — always**	`SUBMITTED` = funds are with Boku
`RETURNED` from `SUBMITTED`	**Yes — always**	Rail reversed after Boku held funds
`SUBMITTED_UNCONFIRMED → FAILED`	**Yes — after lookup**	Do not refund while outcome unknown

The `FUND_PULLING` insight:

Before: need a `funds_pulled` boolean, checked in the refund worker.

After: reaching `SUBMITTED` *is* the guarantee. Refund worker does exactly one thing.

> **REFUND_FAILED** — money is with Boku, held by neither party. Zero-tolerance alert: page oncall immediately, no acknowledgement window.

Refund Design — `REFUND_REQUIRED` + Batch Job

Updated state machine for refunds:

```
FAILED / RETURNED → REFUND_REQUIRED → REFUND_PENDING → REFUNDED | REFUND_FAILED
                          ↑                    ↑
                  Disbursement Worker    Refund Batch Job
                  writes immediately     picks up on schedule
```

The trade-off — User Satisfaction vs System Performance:

Dimension	Immediate (event-driven)	Batch Job via `REFUND_REQUIRED` (chosen)
User satisfaction	Faster refund — merchant gets money back sooner	Delay of up to one batch interval (e.g. 5 min)
System risk	Burst of failures = burst of refund rail calls	Controlled rate — batch job throttles calls
Observability	State scattered across Kafka consumer metrics	`SELECT WHERE status='REFUND_REQUIRED'` — everything visible
Failure recovery	Kafka DLQ — harder to inspect and replay	DB rows are durable, queryable, trivially reprocessable
Concurrency	Consumer race on same payout	`SELECT FOR UPDATE SKIP LOCKED` — safe, no double-execution

Safe batch pickup:

```sql
SELECT payout_id, version FROM payouts
WHERE  status = 'REFUND_REQUIRED' ORDER BY updated_at ASC LIMIT 100
FOR UPDATE SKIP LOCKED;
-- advance to REFUND_PENDING before the rail call
```

Cancellation — Boundary at the First External Call

Rule: `POST /payouts/{id}/cancel` is only permitted **before any external API call has been made**.

Status	Cancellable?	Why
`PENDING`	✅ Yes	DB insert + Kafka only — no external call
`PENDING_COMPLIANCE`	✅ Yes	Compliance is Boku-internal — no partner contacted
`PENDING_MANUAL_REVIEW`	✅ Yes	Human queue — safe to withdraw
`FUND_PULLING`	❌ No	Fund pull API already called — unknown mid-flight state
`SUBMITTED` and beyond	❌ No	Disbursement rail called — use refund flow instead
Any terminal state	❌ No	Already resolved — return `422`

Response on non-cancellable state — `422`, not `409`:

```
{ "error_code": "CANCELLATION_NOT_ALLOWED",
  "message": "Payout is in FUND_PULLING. Cancellation not permitted after external API call.
              Funds will be refunded automatically if disbursement fails." }
```

`409` implies a retry might work. `422` is unambiguous: the window is closed.

Race condition — cancel vs. compliance-pass arriving simultaneously:

```
UPDATE payouts SET status='CANCELLED', version=version+1
```

Idempotency — Three Layers, One Principle

Idempotency-Key vs. Correlation-ID — don't collapse them:

- `Idempotency-Key` — caller-supplied, stable across all retries. *"Is this a duplicate?"*
- `Correlation-ID` — generated per attempt, threaded through logs/events/spans. *"How do I trace this execution?"*

THREE ENFORCEMENT POINTS

Layer	Mechanism
Ingress (caller → API)	`Idempotency-Key` header + Postgres unique constraint in same ACID transaction as INSERT
Internal (Kafka consumers)	`UPDATE ... WHERE status = 'X'` — duplicate event → no-op, not double transition
Egress (service → partner rail)	`client_reference = Idempotency-Key` on every outbound call — enables `SUBMITTED_UNCONFIRMED` recovery

Also covers the 30s API Gateway timeout: caller retries same key → dedup returns original response. No new state or logic needed.

Observability — Audit Table as Foundation

```sql
CREATE TABLE payout_audit (
    id             BIGSERIAL PRIMARY KEY,
    payout_id      VARCHAR(40)  NOT NULL,
    correlation_id VARCHAR(40)  NOT NULL,
    from_status    VARCHAR(40),            -- NULL for first PENDING row
    to_status      VARCHAR(40)  NOT NULL,
    triggered_by   VARCHAR(60)  NOT NULL,  -- 'api', 'compliance-service', 'fund-pull-worker'...
    duration_ms    INTEGER,                -- time spent IN this step
    failure_reason VARCHAR(80),
    partner_ref    VARCHAR(120),
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
```

Append-only. One row per state transition. Never updated. The mutable `status` column is a projection — the audit table is the immutable ledger.

What it unlocks: per-step SLA monitoring, P95 latency dashboards, proactive alerting on step duration, full trace of every payout's lifecycle.

Top 5 Alerts (from 18 defined)

Priority	Alert	Condition	Why selected
☑	`POST_PAYOUTS_CRITICAL`	P95 response > 1,000ms for 1min	Only step where every caller is affected simultaneously
☑	`FAIL_RATE_CRITICAL`	> 15% submissions failed in 5min	Systemic rail problem already hitting real volume — every minute compounds
☑	`REFUND_FAILED_ANY`	Any `REFUND_FAILED` in 5min	Zero-tolerance — money with Boku, no ack window, page immediately
☑	`SUBMITTED_UNCONFIRMED_AGE`	Any payout in `SUBMITTED_UNCONFIRMED` > 15min	Silent double-payment risk if failover triggers without resolution
☑	`REFUND_PENDING_AGE`	Any payout in `REFUND_PENDING` > 30min	Caller's money held with no progress — regulatory issue in some jurisdictions

> 15min and 30min thresholds are **policy decisions**, not technical ones. Good to surface live — what is Boku's tolerance?

AWS Architecture

Component	Service	Why
Public API entry	**API Gateway**	Auth, rate limiting, request validation
Payout services	**EKS**	6 Spring Boot microservices: Payout Service, Compliance Service, Fund Pull Worker, Disbursement Worker, Refund Worker, Webhook Dispatcher
Event backbone	**MSK (Kafka)**	6 topics — durable, ordered, replayable
System of record	**RDS PostgreSQL**	ACID for payout ledger + `payout_audit` + idempotency-key table
Retry / DLQ	**SQS + DLQ**	Per-rail exponential backoff; exhausted messages for follow-up
Idempotency fast-path	**ElastiCache (Redis)**	Sub-ms duplicate-key check before hitting Postgres
Scheduled reconciliation	**Lambda + EventBridge**	Matches partner reports against `payout_audit`
Observability	**CloudWatch + X-Ray**	Per-step duration metrics, distributed traces keyed on Correlation ID

Language/framework: Java 21 + Spring Boot (matches Boku stack). WebFlux for Disbursement Worker and Fund Pull Worker — high fan-out to slow rails is the non-blocking use case it's built for.

Stakeholder Coverage — Honest Assessment

Role	Covered	Gap
Customer	Idempotency, dual pull+push tracking, structured error codes, full history	No stated SLA on `PENDING_COMPLIANCE` wait time
Developer	Full state machine, Correlation-ID vs Idempotency-Key split, consistent error shape	No test strategy, no API versioning scheme
DevOps	Full AWS mapping, 6 EKS services, MSK/RDS/SQS/Secrets Manager	No canary/blue-green strategy, no autoscaling policy
Operation Team	Audit table, 5 priority alerts with thresholds, DLQ for retries	No admin API; runbooks are stubs — data supports them, procedures not written
Financial Team	Append-only audit trail, reconciliation design, REFUNDED trail	No GL integration point, no retention policy
Stakeholder	Greenfield rationale, AWS choices with precedent	No TCO estimate, no phased timeline

Strongest on: Customer / Developer / DevOps — direct extensions of DASH-proven patterns.
Thinnest on: Financial Team / Stakeholder — name these gaps first, don't wait to be caught.

Open Questions for the Walkthrough

Q1 — Managed service or connectivity layer?
If caller specifies the rail → rail selection layer gone, failover-blocking in `SUBMITTED_UNCONFIRMED` simplified. Changes the entire §3.9 story.

Q2 — If platform picks rail: priority-ordered fallback or multi-rail scoring?

- Priority-ordered fallback → circuit breaker model, current design applies
- Multi-rail scoring → need a selection step not currently designed

Q3 — Prefunded wallet or Pull-and-Pay?
If prefunded → remove `FUND_PULLING` / `FUND_PULL_FAILED` / refund flow; add wallet debit in acceptance transaction; synchronous `INSUFFICIENT_FUNDS`. Show you know both.

Demo

Live API Playground

`https://boku-payouts-design.vercel.app/api-docs/?server=https://mywiremockserver-production.up.railway.app`

DEMO IDS TO TRY

- `pyo_SETTLED_001` — happy path with full history + timing
- `pyo_FAILED_001` — `BENEFICIARY_ACCOUNT_INVALID`
- `pyo_UNCONF_001` — `SUBMITTED_UNCONFIRMED`
- `btc_DEMO_001` — batch: 8 settled / 2 failed
- `qte_EXPIRED_001` — expired FX quote